

SOLUTION ARCHITECT PRESENTATION · 2025

VOICE VAULT AI

Context-Aware Voice Privacy Assistant

An AI-powered platform that transcribes voice input, detects PII privacy risks in real-time, and delivers spoken assessments — built on Clean Architecture.

Solution Architect | Architecture Review · 2025



What We'll Cover

01

Problem Statement

Why voice privacy is broken

02

Solution Overview

WhisperShield AI value proposition

03

System Architecture

Clean Architecture layers

04

Core Components

Key building blocks

05

Technology Stack

Frameworks, SDKs, and infra

06

Privacy Engine

PII detection and risk scoring

07

Data Flow

End-to-end processing pipeline

08

Roadmap & Next Steps

Phases and action items

Voice Notes Carry Hidden Privacy Risks

Every time someone records a voice note — dictating a message, leaving a memo, or dictating sensitive information — they may unknowingly include PII that gets uploaded, stored, and exposed without any privacy check.

PII Guardrails

Traditional voice tools have zero awareness of sensitive content. Passwords, SSNs, health data, and credit card numbers are transcribed and stored with no warning.

Invisible Risk Exposure

Users cannot see what sensitive entities their voice note contains before it leaves the device. Risk is invisible until a breach occurs.

Zero Context Awareness

Voice assistants treat each recording in isolation — no intent understanding, no privacy-aware routing, no adaptive response.

Compliance Gaps

Organisations processing voice data lack automated PII detection, making GDPR compliance and data minimisation principles hard to enforce.

67%

users frustrated by repeating PII

\$4.5M

average cost of a data breach
(IBM, 2023)

3x

more likely to abandon voice apps
after privacy issues

40%

of voice recordings contain
sensitive personal info

VOICE VAULT AI — Privacy-First Voice Analysis

An intelligent voice privacy assistant that transcribes spoken input via Azure Whisper, evaluates the transcript for PII risks using a domain-driven privacy engine, and returns a spoken risk assessment — all within a Clean Architecture.



Voice-Native

Records audio directly in the browser via Streamlit's audio component. No external audio pipeline required.



Azure Whisper STT

High-accuracy multilingual transcription via Azure OpenAI Whisper — streaming-capable with async Python.



Privacy Engine

Keyword-weighted PII detector evaluates transcript for passwords, SSNs, health data, emails, and more with scored risk levels.



Voice Response

gTTS converts the privacy assessment back into natural speech — results are both displayed and spoken.



Clean Architecture

Domain, Application, Infrastructure, and Presentation layers are fully decoupled. Easily swap STT provider or TTS engine.

Clean Architecture Layers

PRESENTATION LAYER

Streamlit Frontend (app.py) + FastAPI Backend (api.py)

User interacts via browser. FastAPI exposes POST /analyze-privacy endpoint. Streamlit renders results and plays TTS audio.

APPLICATION LAYER

PrivacyService (privacy_service.py) + ITranscriptionService (interfaces.py)

Orchestrates: transcribe audio → evaluate privacy. Depends only on abstractions — no infrastructure knowledge.

DOMAIN LAYER

PrivacyEvaluator + PrivacyAssessment + PrivacyEntity + RiskLevel

Pure Python domain logic. Keyword-weighted PII scoring. No external dependencies. Fully testable in isolation.

INFRASTRUCTURE LAYER

AzureWhisperService (azure_whisper_stt.py) + SecurityMiddleware (security.py)

Implements ITranscriptionService using Azure OpenAI SDK. API token validation via FastAPI Header dependency.

Key Building Blocks

Voice Capture Module

- › Streamlit `st.audio_input()` captures raw audio from the browser microphone
- › Returns WAV bytes — zero external audio pipeline required
- › Works in-browser: no native app needed

STT — Azure Whisper

- › `AzureWhisperService` implements `ITranscriptionService` interface
- › Uses `AsyncAzureOpenAI` client (`async/await`)
- › Falls back to mock transcript when credentials absent
- › Config via env vars: `AZURE_OPENAI_API_KEY`, `ENDPOINT`, `DEPLOYMENT_ID`

Privacy Engine (Domain)

- › `PrivacyEvaluator.evaluate(text)` — pure static method, no I/O
- › Scans for 8 PII keyword categories (password, SSN, credit card, health...)
- › Weighted risk scores normalised to 0.0–1.0
- › Returns `PrivacyAssessment` with entities, risk level, recommendation

Risk Classification

- › CRITICAL (≥ 0.8): DO NOT UPLOAD
- › HIGH (≥ 0.5): Redaction recommended
- › MEDIUM (≥ 0.2): Minor risks detected
- › LOW (< 0.2): Safe to process

TTS Response

- › `gTTS` converts full assessment summary to MP3 audio
- › Plays back automatically in the Streamlit UI
- › Summary includes: transcript + entities + risk level + recommendation

API Security

- › FastAPI Header dependency: `validate_api_token()`
- › X-API-Token header required on all protected endpoints
- › HTTP 401 if missing, HTTP 403 if invalid
- › Token configured via `WHISPER_SHIELD_API_TOKEN` env var

Technology Stack

FRONTEND



Streamlit ≥ 1.24

Rapid UI: audio input, results, TTS playback

BACKEND API



FastAPI + Uvicorn

Async REST: POST /analyze-privacy, GET /health

DATA VALIDATION



Pydantic v2

Request/response models with strict types

SPEECH-TO-TEXT



Azure OpenAI Whisper

High-accuracy async transcription via SDK

PRIVACY ANALYSIS



Custom Domain Logic

Keyword-weighted PII scorer — pure Python

TEXT-TO-SPEECH



gTTS ≥ 2.3

Google TTS for spoken assessment readback

RUNTIME



Python 3.10+

Async/await throughout the stack

CLOUD PROVIDER



Azure (OpenAI)

Whisper deployment on Azure OpenAI Service

AUTH



API Token Header

X-API-Token via FastAPI Header dependency

PACKAGE MANAGER



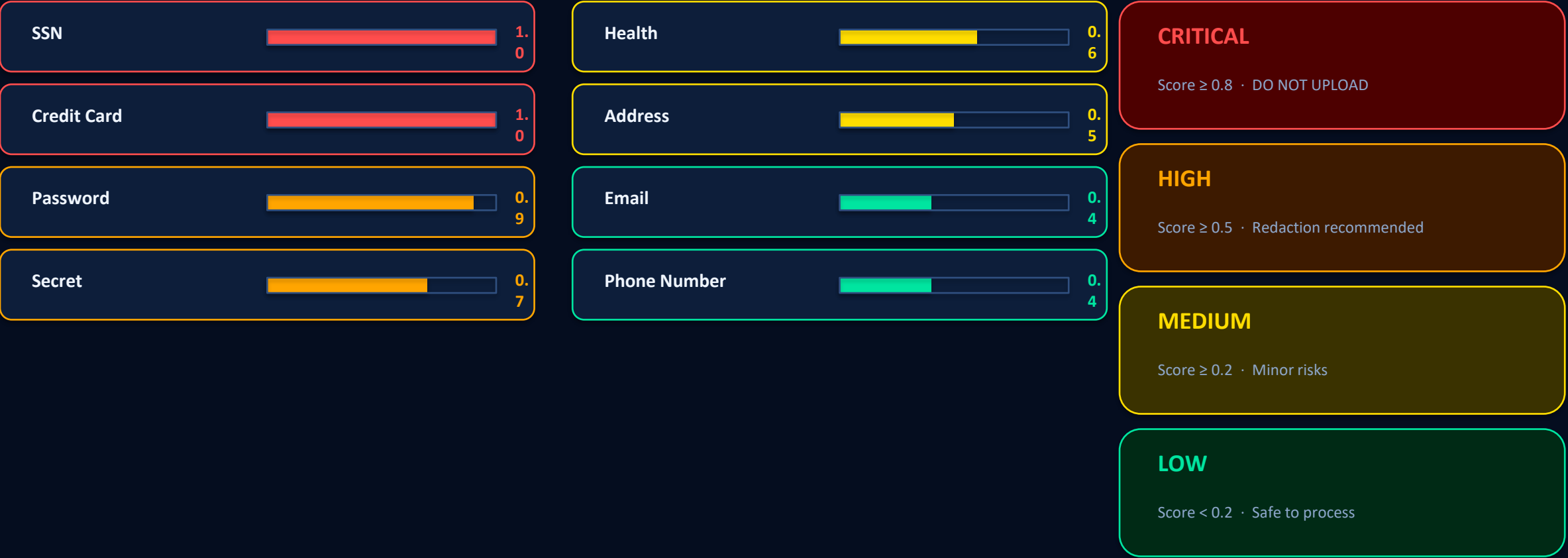
pip / requirements

fastapi, uvicorn, pydantic, openai, gtts

PII Detection & Risk Scoring

Sensitive Keyword Registry (PrivacyEvaluator.SENSITIVE_KEYWORDS)

Risk Level Thresholds



`score = min( $\sum$  keyword_weights, 1.0)     $\rightarrow$     level = CRITICAL if score  $\geq 0.8$  | HIGH if  $\geq 0.5$  | MEDIUM if  $\geq 0.2$  | LOW otherwise`

End-to-End Processing Pipeline

1

User Records Audio

Browser microphone → Streamlit st.audio_input()

Presentation Layer

2

Audio Streamed to Backend

POST /analyze-privacy · multipart/form-data WAV bytes · X-API-Token header validated

FastAPI + Security

3

Azure Whisper Transcription

AzureWhisperService.transcribe(audio_buffer) → async call to Azure OpenAI Whisper deployment

Infrastructure Layer

4

Privacy Risk Analysis

PrivacyService.analyze_audio_privacy() → PrivacyEvaluator.evaluate(transcript)

Application + Domain

5

Risk Assessment Returned

PrivacyAssessment: transcript + detected_entities + risk_level + score + recommendation

Domain Model (Pydantic)

6

Results Displayed + Spoken

Streamlit renders entity table, risk badge, score metric → gTTS synthesises spoken summary

Presentation Layer

Security & Privacy Design

API Token Auth

X-API-Token header on every request. 401 if absent. 403 if invalid. Configured via env var.

PII Detection

Transcript scanned for 8 sensitive keyword classes before any storage or further processing.

HTTPS / TLS

All client-backend traffic encrypted in transit. FastAPI behind HTTPS-terminating reverse proxy in production.

Env-Based Secrets

API keys, endpoints, and tokens loaded from environment variables — never hardcoded in source.

No Persistent Storage

Current MVP performs in-memory analysis only. Audio and transcripts are not persisted — minimising data exposure.

Extensible Compliance

Architecture supports adding audit logging, data retention TTLs, and GDPR right-to-erasure in future phases.

Mock Dataset & NLU Foundation

The project ships a structured intent-classification dataset for training, testing, and validating the NLU layer of the voice assistant.

datasets/clean/

- basic_queries.json
- navigation.json
- reminder.json
- weather.json

datasets/location/

- geofence.json
- gps_drift.json
- indoor.json
- low_accuracy.json

datasets/multimodal/

- audio_location.json
- time_location.json
- weather_location.json

datasets/noisy_asr/

- accent.json
- background_noise.json
- spelling_error.json

Sample Schema Fields

- › id · category · query · intent · entities
- › modality (audio/text/visual/multimodal)
- › confidence (0.0–1.0) · response · action

Demo PII Test Files: Finincial_Assessment_of_Alpha.txt (banking, SSN, credit card data) · Health_Assessment_of_Alpha.txt (medical, health, personal data) → used to trigger CRITICAL/HIGH risk scores

Implementation Roadmap

PHASE 1

Foundation

0 – 2 mo

- › Provision Azure OpenAI (Whisper deployment)
- › FastAPI scaffold + /analyze-privacy endpoint
- › Streamlit frontend with audio input
- › API token security middleware
- › Local dev environment + env var config

PHASE 2

Privacy Intelligence

2 – 4 mo

- › Expand PII keyword registry (IBAN, passport, DOB...)
- › NER-based entity detection (spaCy / Presidio)
- › Streaming Whisper for lower latency
- › Structured audit log per request
- › Unit + integration test suite

PHASE 3

Context & Memory

4 – 6 mo

- › Session context: remember prior utterances
- › Redis-backed short-term memory
- › Vector DB for long-term user profile
- › RAG-based contextual response enrichment
- › Multi-turn conversation support

PHASE 4

Scale & Comply

6 – 9 mo

- › Kubernetes deployment (AKS / EKS)
- › Multi-language STT + TTS support
- › GDPR right-to-erasure automation
- › Observability: traces, metrics, dashboards
- › GA release + hypercare support

Recommended Next Steps

1

Configure Azure OpenAI Credentials

Set `AZURE_OPENAI_API_KEY`, `AZURE_OPENAI_ENDPOINT`, and `AZURE_OPENAI_WHISPER_DEPLOYMENT_ID` environment variables to activate real Whisper transcription (currently in mock mode).

2

Run End-to-End Demo

`pip install -r requirements.txt` → launch FastAPI backend (uvicorn) → launch Streamlit frontend → record voice reading from `Health_Assessment_of_Alpha.txt` → observe CRITICAL risk output.

3

Expand PII Keyword Registry

Add IBAN, passport numbers, date-of-birth, and address patterns to PrivacyEvaluator. Consider replacing keyword matching with spaCy NER or Microsoft Presidio for higher accuracy.

4

Add Session Context Layer

Integrate Redis for short-term conversation memory. Store previous transcripts and risk assessments per session so the assistant can reference prior interactions.

5

Harden Security for Production

Replace static API token with OAuth 2.0 / JWT. Add HTTPS termination, rate limiting, structured audit logging, and a secrets manager (Azure Key Vault / AWS Secrets Manager).

WHISPERSHIELD AI · SOLUTION ARCHITECTURE · 2025

Thank You

Questions & Discussion

Slides, architecture diagrams, and source code available on request.

 solution.architect@company.com

 [GitHub: Context-Aware-Voice-Assistance](#)

 [Schedule a follow-up](#)